



Document de synthèse

QuartierConnect — réalisation du projet annuel

1. Contexte et objectif

QuartierConnect est un réseau social de proximité : une plateforme qui réunit les résidents d'un même quartier autour du signalement d'incidents, de l'entraide entre voisins (services valorisés en points), des événements locaux, d'une messagerie temps réel, de votes communautaires et de la signature électronique de contrats. L'objectif était de livrer une application **complète, sécurisée et réellement déployée**, pas une simple maquette.

Le produit final se décline en **trois surfaces reliées par un compte unique (SSO)** — un client résident et un back-office d'administration en React 19, plus une application desktop JavaFX de synchronisation hors-ligne — au-dessus d'une **API NestJS unique** et d'une **persistance polyglotte** (PostgreSQL, MongoDB, Neo4j, cache SQLite). L'application est **en production** sur quartierconnect.duckdns.org.

2. Démarche de réalisation

› 2.1 Cadrage et analyse du sujet

Le projet a démarré par la lecture du cahier des charges et la définition du périmètre fonctionnel. Nous avons volontairement retenu un périmètre **large mais cohérent** (13 domaines fonctionnels côté résident), en gardant un fil conducteur : tout ce qui touche à la vie d'un quartier, cloisonné géographiquement.

› 2.2 Choix d'architecture

Trois décisions structurantes ont été prises tôt :

- **Monorepo** (Turborepo + pnpm workspaces) pour mutualiser le design system, les hooks, l'internationalisation et le client HTTP entre le client et l'admin, tout en gardant des builds indépendants.
- **Persistance polyglotte** : chaque type de donnée est confié à la base la plus adaptée — PostgreSQL (ACID) pour l'identité, l'authentification, les incidents et le registre de points ; MongoDB (documents + GridFS) pour le contenu métier (quartiers, services, événements,

messaging, contrats, médias) ; Neo4j (graphe) pour le réseau social et les recommandations ; SQLite pour le cache local du desktop.

- **Une API unique, trois surfaces** : le même back-end sert le web, l'admin et le desktop, avec un SSO par jeton éphémère à usage unique.

› 2.3 Méthode et organisation

Le travail a été mené de façon **itérative**, domaine par domaine. Chaque évolution passait par une **branche + pull request** avec revue, et une **intégration continue obligatoire** (lint, typecheck, tests, build) avant fusion. Des conventions de commit et des git hooks partagés garantissaient l'homogénéité du dépôt. La commande `make setup` permet de reconstruire tout l'environnement en une seule commande, ce qui a fiabilisé le travail à plusieurs.

› 2.4 Grandes phases

PHASE	CONTENU
Fondations	Authentification (JWT + 2FA TOTP), schéma des trois bases, squelette monorepo, chaîne CI.
Domaines fonctionnels	Incidents, services, événements, points, votes, messaging, contrats — un domaine après l'autre.
Temps réel & desktop	Messaging Socket.IO ; application JavaFX avec synchronisation hors-ligne.
Sécurité	Durcissement complet (rotation stricte des tokens, anti-rejeu TOTP, CSP par route, fail2ban), audit interne.
Qualité	Tests unitaires (Jest, Vitest, TestFX) et E2E (Playwright), seuils de couverture en CI.
Déploiement	Conteneurisation Docker, reverse proxy Caddy, déploiement VPS automatisé avec smoke test et rollback.
Documentation	Sites d'aide et développeur (Fumadocs) + référence API Scalar générée depuis le code.
Finition	Refonte visuelle « service public » (identité bleu France), corrections et perfectionnement avant rendu.

› 2.5 Outillage

Docker Compose (9 conteneurs), Caddy (TLS + CSP), Turborepo + pnpm, GitHub Actions (CI, E2E, déploiement SSH, publication des installeurs `jpackage`), Playwright / Jest / Vitest / JUnit-TestFX, ESLint + Prettier, migrations Drizzle, et un DSL d'administration maison écrit en Python (PLY).

3. Répartition du travail

Répartition du travail au sein de l'équipe :

AXE	CLAUDIO REIBAUD	MOUHAMADOU N'DIAYE	ANDRAS SCHULLER
Produit, UX & design	•		
Front web (client + admin React)	•		
API / back-end (NestJS, bases)	•		
Application desktop (JavaFX, sync)	•		
DevOps / CI-CD / déploiement	•	•	
Sécurité (2FA, tokens, durcissement)	•		
Tests (unitaires, E2E)	•		•
Documentation	•	•	•
DSL	•		

(• = *contribution*.)

Note. Une partie du développement a été menée **en binôme**, les sessions étant poussées depuis un seul poste : l'historique Git ne reflète donc pas fidèlement la contribution individuelle de chacun. La répartition ci-dessus rend compte du travail réel de l'équipe.

4. Analyse critique et objective

› 4.1 Points forts

- **Une architecture soignée et justifiée** : la persistance polyglotte n'est pas gratuite — chaque base est choisie pour la nature de sa donnée, et une seule couche (l'API) y accède.
- **Un niveau de sécurité proche du monde réel** : Argon2id sur les mots de passe *et* les refresh tokens, 2FA TOTP avec anti-rejeu, rotation stricte des tokens sous transaction, CSP par route et fail2ban.
- **Une vraie couverture de tests** (~149 fichiers) et une **CI/CD complète** avec déploiement automatisé, smoke test et rollback.
- **Des choix originaux qui se démarquent** : synchronisation desktop *offline-first* par fusion à trois voies (façon Git), et un DSL d'administration maison.
- **Un produit réellement déployé** et fonctionnel de bout en bout, avec sa documentation générée depuis le code.

› 4.2 Difficultés rencontrées et solutions

- **Cohérence entre trois bases** : le seed, les sauvegardes et les jeux d'essais ont dû être pensés pour les trois moteurs à la fois → un script d'import/seed unique et des sauvegardes coordonnées.
- **Synchronisation hors-ligne** : gérer les modifications concurrentes entre le poste local et le serveur → un *merge* à trois voies champ par champ, avec résolution manuelle des conflits et suppressions par *tombstones*.
- **Temps réel** : authentifier les WebSockets et diffuser présence / « écrit... » sans *polling* → Socket.IO adossé au JWT, avec des événements dédiés.
- **Déploiement mono-hôte** : faire cohabiter neuf conteneurs et trois bases sur un seul VPS → limites mémoire par service, TLS automatique via Caddy, et pipeline de déploiement avec rollback pour ne jamais laisser la production cassée.

› 4.3 Limites et dette technique

De façon honnête :

- La **montée en charge** n'a pas été éprouvée : un seul VPS, pas de scaling horizontal ni de test de charge formel.
- La **couverture de tests du desktop** reste plus légère que celle du back-end.

- Il n'y a **pas d'application mobile native** (le web est responsive, mais ce n'est pas la même chose).
- Certaines fonctions avancées (vote pondéré, plugins desktop) sont **complètes mais peu exercées** en démonstration.
- Les **sauvegardes** existent et sont testées, mais aucun **plan de reprise d'activité** formel n'a été rédigé.

› 4.4 Améliorations envisagées

Notifications *push* (mobile et desktop), observabilité (métriques et traces), tests de charge, renforcement de la CI de la partie desktop, et ouverture **multi-quartiers** à plus grande échelle.

5. Conclusion

QuartierConnect nous a permis de mener un projet **de bout en bout** : du cahier des charges à une application déployée en production, en passant par des choix d'architecture argumentés, une sécurité sérieuse, une chaîne de tests et de livraison automatisée, et une documentation complète. Les principales limites identifiées (montée en charge, mobile natif) sont clairement des **pistes de poursuite**, pas des impasses — ce qui, pour un projet annuel, nous semble un bon équilibre entre ambition et maîtrise.